

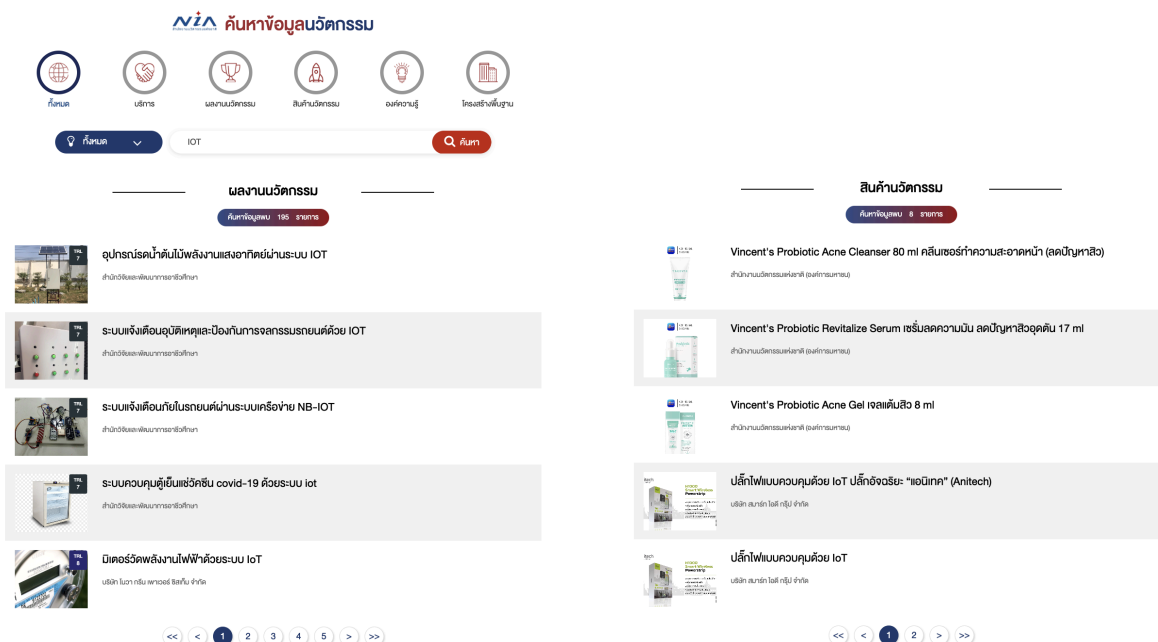
Implementation Project: Improving Search Results for a Website

For this project, I want to focus on a real-world issue at my current workplace, the National Innovation Agency (NIA) of Thailand, which sponsors my studies. NIA is a government organization committed to fostering innovation by providing support to individuals, SMEs, and startups.

One of the projects I previously worked on is the <https://www.thailandinnovationportal.com/>. The website's objective is to provide valuable information to help users advance their innovation efforts. This website serves as a hub for innovation-related information, aimed at empowering users to make progress in their ventures. While the website successfully launched and provides valuable content, there is still considerable potential for improvement, particularly in the search functionality.

Problem Identification

Initially, I considered redesigning the overall structure of the website. However, after reviewing the user experience, I found that the search function was a more pressing issue. For example, searching for "IoT" (<https://www.thailandinnovationportal.com/info/search>) yields mixed results: while the first section seems relevant, the subsequent sections often include irrelevant entries. This behavior likely stems from the search algorithm prioritizing character matches over semantic relevance.



Given the importance of the search function for users navigating a large database of innovation-related content, I decided to focus on enhancing this aspect. My goal is to implement a more efficient and user-friendly search mechanism, inspired by concepts from the coursework on search indexing and ranking, as well as in natural language processing (NLP) and word tokenization.

Project Goals and Approach

The primary objective is to improve the search algorithm to ensure that user queries retrieve the most relevant documents with higher accuracy. Each row of the dataset represents a document, and the system should rank these documents based on their similarity to the query.

1. Data Preprocessing

The first step involved acquiring the actual data used by the website. After consulting with colleagues, I gained access to the database that hosts the data. However, the data format required significant preprocessing. The information was stored across multiple tables, necessitating data cleaning and merging to create a usable dataset.

The dataset contains a wide range of columns, but I focused on those most relevant to the search function: **Title**, **Detail**, and **Highlight**. These columns represent the key content users are likely to search. Titles and highlights are concise and keyword-rich, while details offer more descriptive and supplementary information.

2. Natural Language Processing and Bag of Words

To enhance the search functionality, I used NLP techniques to tokenize and analyze the text. Most of the data is in Thai, presenting a unique challenge due to the lack of spaces between words in the language. For tokenization, I utilized the PyThaiNLP library (<https://pythainlp.org/>, <https://pythainlp.org/docs/5.0/>), which is specifically designed for processing Thai text. I use two engines, **sefr_cut** and **tltk**, in combination to achieve optimal tokenization results.

 / pythainlp.tokenize [View page source](#)

pythainlp.tokenize

The `pythainlp.tokenize` module contains a comprehensive set of functions and classes for tokenizing Thai text into various units, such as sentences, words, subwords, and more. This module is a fundamental component of the PyThaiNLP library, providing tools for natural language processing in the Thai language.

```
pythainlp.tokenize.word_tokenize(text: str, custom_dict: ~pythainlp.util.trie.Trie = <pythainlp.util.trie.Trie object>, engine: str = 'newmm', keep_whitespace: bool = True, join_broken_num: bool = True) → List[str] \[source\]
```

Word tokenizer.

Tokenizes running text into words (list of strings).

Parameters:

- **text** (*str*) – text to be tokenized
- **engine** (*str*) – name of the tokenizer to be used
- **custom_dict** (*pythainlp.util.Trie*) – dictionary trie (some engine may not support)
- **keep_whitespace** (*bool*) – True to keep whitespace, a common mark for end of phrase in Thai. Otherwise, whitespace is omitted.
- **join_broken_num** (*bool*) – True to rejoin formatted numeric that could be wrongly separated. Otherwise, formatted numeric could be wrongly separated.

Returns: list of words

Return type: List[str]

Options for engine

- *attacut* - wrapper for [AttaCut](#)., learning-based approach
- *deepcut* - wrapper for [DeepCut](#), learning-based approach
- *icu* - wrapper for a word tokenizer in [PyICU](#)., from ICU (International Components for Unicode), dictionary-based
- *longest* - dictionary-based, longest matching
- *mm* - "multi-cut", dictionary-based, maximum matching
- *nercut* - dictionary-based, maximal matching, constrained by Thai Character Cluster (TCC) boundaries, combining tokens that are parts of the same named-entity
- *newmm* (default) - "new multi-cut", dictionary-based, maximum matching, constrained by Thai Character Cluster (TCC) boundaries with improved TCC rules that are used in newmm.
- *newmm-safe* - newmm, with a mechanism to avoid long processing time for text with continuously ambiguous breaking points
- *nlp03* - wrapper for a word tokenizer in [nlp03](#)., adaptation of newmm in Rust (2.5x faster)
- *oskut* - wrapper for [OSKut](#)., Out-of-domain Stacked cut for Word Segmentation
- *sefr_cut* - wrapper for [SEFR CUT](#)., Stacked Ensemble Filter and Refine for Word Segmentation
- *tltk* - wrapper for [TLTK](#).,

Tokenize text with different tokenizers:

```
from pythainlp.tokenize import word_tokenize

text = "โอเคผมพวกรักภรรยาวันเกิด"

word_tokenize(text, engine="newmm")
# output: ['โอเค', 'ผม', 'พวกรัก', 'ภรรยา', 'วันเกิด']

word_tokenize(text, engine='attacut')
# output: ['โอเค', 'ผม', 'พวกรัก', 'ภรรยา', 'วันเกิด']
```

I processed the **Title**, **Detail**, and **Highlight** columns separately, recognizing their differing importance in the context of search. By tokenizing the text, I generated a Bag of Words (BoW) for each document and calculated the total unique words. Additionally, I constructed an inverted index to facilitate efficient search operations.

To manage computational limitations, especially given the time-intensive NLP processing, I restricted the initial scope of the project to 100 rows of data. This subset serves as a proof of concept for further scaling.

3. Search Ranking Algorithm

To improve the relevance of search results, I explored several ranking techniques, including TF-IDF and BM25. While these algorithms provided valuable insights, I opted for a customized approach tailored to the dataset's characteristics:

- **Word Match Scoring:** I assigned scores based on the type of match. Exact word matches received a high score of 10, while partial matches (e.g., substrings) were assigned a lower score of 2. This ensures that exact matches are prioritized without entirely ignoring partial matches.
- **Normalization:** I chose not to account for Term Frequency (TF) or normalize scores using Inverse Document Frequency (IDF). Instead, I assumed that user queries would already be meaningful and relevant. To balance the scores, I normalized them by the number of unique words in each document, effectively prioritizing shorter, keyword-rich sentences.
- **Aggregated Scoring:** For multi-word queries (separated by whitespace), the algorithm calculates scores for each word individually and aggregates them.

4. Implementation of the Search System

To demonstrate the enhanced search functionality, I developed a basic website using HTML and vanilla JavaScript. The subset of data, along with the inverted index and word counts, is stored as JSON files and loaded into the website during runtime.

The website consists of two main pages:

1. **Data Table Page:** This page displays all documents in a tabular format. To improve usability, I implemented pagination, allowing users to navigate large datasets easily.
2. **Search Page:** This page features a search bar where users can input their queries. Upon clicking the search button, the system calculates similarity scores for each document and displays the results in descending order of relevance. (index page)

The website design builds upon a previous class assignment, adapted to focus on the search functionality. While the interface is simple, it effectively demonstrates the core improvements to the search algorithm. Here is the website <https://bhirajaya.github.io/index.html>.

[illegible]

Anticipated Outcomes

By incorporating concepts from search ranking and NLP, I aim to create a more efficient and relevant search system for the Thailand Innovation Portal. This project not only addresses a real-world challenge but also allows me to apply theoretical knowledge to practical problems. In the future, I hope to refine the ranking algorithms to better align with user needs, expand the prototype to encompass the full dataset, and seamlessly integrate the improved search functionality into the live website. Ultimately, these enhancements are expected to significantly improve user experience and contribute to fostering innovation in Thailand by providing more accessible and relevant information.